

Project 2 Embedded Systems Test Log

Author: Sam O'Connor

Student Number: C22316191

Table of Contents

Task 1: Re-establishing the Project 1 System	1
Task 2: Configuring the I ² S Speaker Output.....	12
Task 3: Audio Playback Verification	16
Task 4: Timer-Based Audio Control	21
Task 5: DMA-Based ADC Sampling	27
Task 6: Audio Response Based on Tilt Direction	35
Audio_data.h file	41
Task 7: Final Integration, Optimisation and Testing.....	55

Task 1: Re-establishing the Project 1 System

Goal

Restore the fully working tilt-controlled LED ring system from Project 1.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>

#include "eeng1030_lib.h"
#include "i2c.h"

#define NUM_LEDS    16
#define WS_PIN      7
#define BTN_PIN     9

#define TOP_LED     0
#define DEAD_ZONE   180
#define SAVE_THRESHOLD 200

uint16_t response;
```

```

int32_t X_g, Y_g, Z_g;
int32_t X_filt = 0, Y_filt = 0, Z_filt = 0;

int saved_mode = 0;
int saved_centre_led = 0;
int save_latched = 0;

static const int16_t dir_x[NUM_LEDS] = {
    0, 383, 707, 924, 1000, 924, 707, 383,
    0, -383, -707, -924, -1000, -924, -707, -383
};

static const int16_t dir_y[NUM_LEDS] = {
    1000, 924, 707, 383, 0, -383, -707, -924,
    -1000, -924, -707, -383, 0, 383, 707, 924
};

void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void gpio_init_button(void);
static int button_pressed(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void initADC(void);
static int readADC(int chan);
static void initTimer2PWM(void);
static void setTimer2Duty(int duty);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y, int32_t z);
static void show_live_and_saved(int live_centre_led, int saved_centre_led, int
saved_active, uint8_t g, uint8_t r, uint8_t b);

int main(void)
{
    uint8_t colour_index = 0;

```

```

uint8_t last_button_state = 0;

uint8_t g_val = 0x00;
uint8_t r_val = 0x08;
uint8_t b_val = 0x00;

clock_init_80mhz();
setup();
initSerial(9600);
gpio_init_ws2812();
gpio_init_button();
dwt_init();

pinMode(GPIOA, 0, 3);
pinMode(GPIOA, 3, 2);
GPIOA->AFR[0] &= ~(0xF << (4 * 3));
GPIOA->AFR[0] |= (1U << (4 * 3));

initADC();
initTimer2PWM();

ResetI2C();

I2CStart(0x69, WRITE, 2);
I2CWrite(0x7E);
I2CWrite(0x11);
I2CStop();

delay(1000000);

while (1)
{
    uint8_t current_button_state = button_pressed();

    if ((current_button_state == 1) && (last_button_state == 0))
    {
        delay(30000);

        if (button_pressed() == 1)
        {
            colour_index++;
            if (colour_index >= 4)
                colour_index = 0;

            while (button_pressed() == 1) {}

            delay(30000);
        }
    }
}

```

```

}

last_button_state = current_button_state;

X_g = read_bmi160_axis(0x12);
Y_g = read_bmi160_axis(0x14);
Z_g = read_bmi160_axis(0x16);

X_filt = (3 * X_filt + X_g) / 4;
Y_filt = (3 * Y_filt + Y_g) / 4;
Z_filt = (3 * Z_filt + Z_g) / 4;

int pot_value = readADC(5);
int centre_led = get_pointer_led(X_filt, Y_filt, Z_filt);

if (pot_value <= SAVE_THRESHOLD)
{
    setTimer2Duty(0);

    if (save_latched == 0)
    {
        saved_centre_led = centre_led;
        saved_mode = 1;
        save_latched = 1;
    }
}
else
{
    setTimer2Duty(pot_value);
    saved_mode = 0;
    save_latched = 0;
}

if (colour_index == 0)
{
    g_val = 0x00;
    r_val = 0x08;
    b_val = 0x00;
}
else if (colour_index == 1)
{
    g_val = 0x08;
    r_val = 0x08;
    b_val = 0x00;
}
else if (colour_index == 2)
{
    g_val = 0x02;

```

```

        r_val = 0x08;
        b_val = 0x00;
    }
    else
    {
        g_val = 0x08;
        r_val = 0x00;
        b_val = 0x00;
    }

    printf("POT=%d X=%ld Y=%ld Z=%ld colour=%d saved=%d saved_led=%d
live_led=%d\r\n",
        pot_value, X_filt, Y_filt, Z_filt, colour_index, saved_mode,
saved_centre_led, centre_led);

    show_live_and_saved(centre_led, saved_centre_led, saved_mode, g_val,
r_val, b_val);

    delay(80000);
}
}

static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();
    accel_raw = response;

    response = I2CRead();
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}

static int wrap_led(int n)
{
    while (n < 0) n += NUM_LEDS;

```

```

    while (n >= NUM_LEDS) n -= NUM_LEDS;
    return n;
}

static int get_pointer_led(int32_t x, int32_t y, int32_t z)
{
    int32_t mag2_xy = x * x + y * y;

    if (mag2_xy < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    int best_led = 0;
    int32_t best_dot = -2147483647;

    for (int i = 0; i < NUM_LEDS; i++)
    {
        int32_t dot = x * dir_x[i] + y * dir_y[i];

        if (dot > best_dot)
        {
            best_dot = dot;
            best_led = i;
        }
    }

    return wrap_led(TOP_LED + best_led);
}

static void show_live_and_saved(int live_centre_led, int saved_centre_led, int
saved_active, uint8_t g, uint8_t r, uint8_t b)
{
    int live_a = wrap_led(live_centre_led - 1);
    int live_b = wrap_led(live_centre_led);
    int live_c = wrap_led(live_centre_led + 1);

    int save_a = wrap_led(saved_centre_led - 1);
    int save_b = wrap_led(saved_centre_led);
    int save_c = wrap_led(saved_centre_led + 1);

    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        uint8_t gg = 0x00;
        uint8_t rr = 0x00;
        uint8_t bb = 0x00;
    }
}

```

```

        if (saved_active && (i == save_a || i == save_b || i == save_c))
        {
            gg = 0x00;
            rr = 0x00;
            bb = 0x08;
        }

        if (i == live_a || i == live_b || i == live_c)
        {
            gg = g;
            rr = r;
            bb = b;
        }

        ws2812_send_pixel(gg, rr, bb);
    }

    ws2812_reset();
}

void setup(void)
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initI2C();
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = 8000000 / baudrate;
}

```

```

    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
}

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;

```



```

RCC->CFGR |= RCC_CFGR_SW_PLL;
while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |= (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);
    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |= (3U << (WS_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

static void gpio_init_button(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR &= ~(3U << (BTN_PIN * 2));
    GPIOA->PUPDR |= (1U << (BTN_PIN * 2));
}

static int button_pressed(void)
{
    if ((GPIOA->IDR & (1U << BTN_PIN)) == 0)
        return 1;
    else
        return 0;
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;

```

```

uint32_t ticks = us * (SystemCoreClock / 1000000U);

while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}
        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}

static void initADC(void)
{

```

```

RCC->AHB2ENR |= (1 << 13);
RCC->CCIPR |= (1 << 29) | (1 << 28);
ADC1_COMMON->CCR = ((0b01) << 16) + (1 << 22);

ADC1->CR = (1 << 28);
delay(100);

ADC1->CR |= (1 << 31);
while (ADC1->CR & (1 << 31)) {}

ADC1->CFGR = (1 << 31);
}

static int readADC(int chan)
{
    ADC1->SQR1 = 0;
    ADC1->SQR1 |= (chan << 6);

    ADC1->ISR = (1 << 3);
    ADC1->CR |= (1 << 0);
    while ((ADC1->ISR & (1 << 0)) == 0) {}

    ADC1->CR |= (1 << 2);
    while ((ADC1->ISR & (1 << 3)) == 0) {}

    return ADC1->DR;
}

static void initTimer2PWM(void)
{
    RCC->APB1ENR1 |= (1 << 0);

    TIM2->CR1 = 0;
    TIM2->CCMR2 = (0b110 << 12) | (1 << 11);
    TIM2->CCER |= (1 << 12);

    TIM2->PSC = 79;
    TIM2->ARR = 1000 - 1;
    TIM2->CCR4 = 0;

    TIM2->EGR |= (1 << 0);
    TIM2->CR1 |= (1 << 7);
    TIM2->CR1 |= (1 << 0);
}

static void setTimer2Duty(int duty)
{
    int arrvalue;

```

```

    if (duty < 0)
        duty = 0;

    if (duty > 4095)
        duty = 4095;

    if (duty <= SAVE_THRESHOLD)
    {
        TIM2->CCR4 = 0;
        return;
    }

    arrvalue = (duty * (TIM2->ARR + 1)) / 4095;
    TIM2->CCR4 = arrvalue;
}

```

Result:

Initial system fully functional:

- Accurate tilt detection
- Stable LED output
- Smooth pointer movement
- Reliable performance

Task 2: Configuring the I²S Speaker Output

Goal:

The aim of this test was to configure the STM32L432 to output serial audio data to the external I²S speaker module. This involved setting PA8, PA9 and PA10 to their SAI alternate functions for BCLK, LRC and DIN, configuring the SAI peripheral, and using DMA to transfer audio samples from memory to the SAI data register.

```

#include "stm32l432xx.h"
#include <stdint.h>
#include "audio_data.h"

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF);
void initClocks(void);

```

```

void setup(void);
void initSAI(void);
void initDMA(void);
void delay(volatile uint32_t dly);

int main(void)
{
    setup();
    initSAI();
    initDMA();

    SAI1_Block_A->CR1 |= (1 << 16);

    while (1)
    {
        GPIOB->ODR ^= (1 << 3);
        delay(100000);
    }
}

void setup(void)
{
    initClocks();

    RCC->AHB2ENR |= (1 << 0) | (1 << 1);

    pinMode(GPIOB, 3, 1);
}

void initSAI(void)
{
    // PA8  = BCLK
    // PA9  = LRC
    // PA10 = DIN

    pinMode(GPIOA, 8, 2);
    pinMode(GPIOA, 9, 2);
    pinMode(GPIOA, 10, 2);

    selectAlternateFunction(GPIOA, 8, 13);
    selectAlternateFunction(GPIOA, 9, 13);
    selectAlternateFunction(GPIOA, 10, 13);

    RCC->APB2ENR |= (1 << 21);

    RCC->CR &= ~(1 << 26);
    while (RCC->CR & (1 << 27)) {}
}

```

```

RCC->PLLSAI1CFGR =
    (2 << 27) |
    (37 << 8) |
    (1 << 16);

RCC->CR |= (1 << 26);
while ((RCC->CR & (1 << 27)) == 0) {}

SAI1->GCR = (1 << 4);

SAI1_Block_A->CR1 = 0;
SAI1_Block_A->CR2 = 0;
SAI1_Block_A->SLOTR = 0;
SAI1_Block_A->FRCR = 0;

SAI1_Block_A->CR1 =
    (7 << 5) |
    (3 << 20) |
    (1 << 9);

SAI1_Block_A->SLOTR =
    (1 << 8) |
    (1 << 16) |
    (1 << 17) |
    (1 << 7);

SAI1_Block_A->FRCR =
    (1 << 18) |
    (1 << 16) |
    (15 << 8) |
    (32 - 1);

SAI1_Block_A->CR1 |= (1 << 17);
}

void initDMA(void)
{
    RCC->AHB1ENR |= (1 << 1);

    DMA2_Channel6->CCR &= ~(1 << 0);

    DMA2_Channel6->CMAR = (uint32_t)audio_data;
    DMA2_Channel6->CPAR = (uint32_t)&SAI1_Block_A->DR;
    DMA2_Channel6->CNDTR = AUDIO_LENGTH;

    DMA2_CSELR->CSELR &= ~(0xF << 20);
    DMA2_CSELR->CSELR |= (1 << 20);
}

```

```

DMA2_Channel16->CCR =
    (1 << 11) |
    (1 << 9) |
    (1 << 8) |
    (1 << 7) |
    (1 << 5) |
    (1 << 0);
}

void initClocks(void)
{
    RCC->CR &= ~(1 << 24);
    while (RCC->CR & (1 << 25)) {}

    RCC->PLLCFGR =
        (1 << 25) |
        (1 << 24) |
        (1 << 22) |
        (1 << 21) |
        (1 << 17) |
        (80 << 8) |
        (1 << 0);

    RCC->CR |= (1 << 24);
    while ((RCC->CR & (1 << 25)) == 0) {}

    FLASH->ACR &= ~0x7;
    FLASH->ACR |= 4;

    RCC->CFGR &= ~0x3;
    RCC->CFGR |= 0x3;

    while ((RCC->CFGR & (0x3 << 2)) != (0x3 << 2)) {}

    SystemCoreClock = 80000000;
}

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    Port->MODER &= ~(3U << (BitNumber * 2));
    Port->MODER |= (Mode << (BitNumber * 2));
}

void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF)
{
    if (BitNumber < 8)
    {

```

```

        Port->AFR[0] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[0] |= (AF << (4 * BitNumber));
    }
    else
    {
        BitNumber -= 8;
        Port->AFR[1] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[1] |= (AF << (4 * BitNumber));
    }
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

```

Result:

The speaker output was successfully configured using SAI and DMA. The audio sample buffer was transferred automatically to the SAI peripheral, while the main loop continued running and toggled PB3 as a debug indicator. This confirmed that the audio output was operating in the background using DMA rather than being manually written by the CPU.

Task 3: Audio Playback Verification

The goal of this task was to verify that audio could be played through the MAX98357A I2S mono amplifier and speaker using the STM32L432. The same I2S wiring was used, with BCLK connected to PA8, LRC connected to PA9 and DIN connected to PA10. The amplifier was powered from 5 V, with SD tied high to enable the module.

The first test used a simple generated waveform stored as an array in memory. This allowed the SAI and DMA setup to be checked before using a real audio file. DMA2 Channel 6 was configured to continuously transfer the audio sample buffer to the SAI1 data register. Circular mode was used so that playback repeated continuously without needing manual reloading inside the main loop.

A heartbeat GPIO output on PB3 was also toggled in the main loop to confirm that the program was still running while the audio was being streamed by DMA. Different buffers were tested, including simple tone data and a converted WAV audio sample. The speaker output confirmed that the amplifier was receiving digital audio data, although some tests produced noise when the sample format or timing did not match the expected I2S format. This helped identify that correct sample formatting, stereo duplication and DMA transfer length were important for successful playback.


```

#include "stm32l432xx.h"
#include <stdint.h>
#include "audio_data.h"

/*
Task 3 audio playback verification

wiring:
MAX98357 VIN   -> 5V
MAX98357 GND   -> GND
MAX98357 SD    -> 5V
MAX98357 BCLK  -> PA8
MAX98357 LRC   -> PA9
MAX98357 DIN   -> PA10

PB3 is used as a heartbeat output
*/

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF);
void initClocks(void);
void setup(void);
void initSAI(void);
void initDMA(void);
void delay(volatile uint32_t dly);

int main(void)
{
    setup();

    initSAI();

    initDMA();

    // start SAI after DMA is ready
    SAI1_Block_A->CR1 |= (1 << 16);

    while (1)
    {
        // heartbeat LED/debug pin
        GPIOB->ODR ^= (1 << 3);
        delay(100000);
    }
}

void setup(void)

```

```

{
    initClocks();

    // enable GPIOA and GPIOB clocks
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);

    // PB3 heartbeat output
    pinMode(GPIOB, 3, 1);
}

void initSAI(void)
{
    // PA8 = BCLK
    // PA9 = LRC / LRCLK
    // PA10 = DIN

    pinMode(GPIOA, 8, 2);
    pinMode(GPIOA, 9, 2);
    pinMode(GPIOA, 10, 2);

    selectAlternateFunction(GPIOA, 8, 13);
    selectAlternateFunction(GPIOA, 9, 13);
    selectAlternateFunction(GPIOA, 10, 13);

    // enable SAI1 clock
    RCC->APB2ENR |= (1 << 21);

    // disable PLLSAI1 before configuring
    RCC->CR &= ~(1 << 26);
    while (RCC->CR & (1 << 27)) {}

    // configure SAI PLL
    RCC->PLLSAI1CFGR =
        (2 << 27) |
        (37 << 8) |
        (1 << 16);

    // enable PLLSAI1
    RCC->CR |= (1 << 26);
    while ((RCC->CR & (1 << 27)) == 0) {}

    // SAI global config
    SAI1->GCR = (1 << 4);

    // clear SAI block registers
    SAI1_Block_A->CR1 = 0;
    SAI1_Block_A->CR2 = 0;
    SAI1_Block_A->SLOTR = 0;
}

```

```

SAI1_Block_A->FRCR = 0;

// 16 bit audio data, master transmitter, clock divider
SAI1_Block_A->CR1 =
    (7 << 5) |
    (3 << 20) |
    (1 << 9);

// two 16 bit slots, left and right
SAI1_Block_A->SLOTR =
    (1 << 8) |
    (1 << 16) |
    (1 << 17) |
    (1 << 7);

// 32 bit frame, 16 bit frame sync
SAI1_Block_A->FRCR =
    (1 << 18) |
    (1 << 16) |
    (15 << 8) |
    (32 - 1);

// enable DMA request from SAI
SAI1_Block_A->CR1 |= (1 << 17);
}

void initDMA(void)
{
    // enable DMA2
    RCC->AHB1ENR |= (1 << 1);

    // disable channel before setup
    DMA2_Channel6->CCR &= ~(1 << 0);

    // memory source is audio sample array
    DMA2_Channel6->CMAR = (uint32_t)audio_data;

    // peripheral destination is SAI data register
    DMA2_Channel6->CPAR = (uint32_t)&SAI1_Block_A->DR;

    // AUDIO_LENGTH already includes duplicated left/right samples
    DMA2_Channel6->CNDTR = AUDIO_LENGTH;

    // DMA2 channel 6 request selection
    DMA2_CSELR->CSELR &= ~(0xF << 20);
    DMA2_CSELR->CSELR |= (5 << 20);

    // DMA setup:

```

```

// memory size = 16 bit
// peripheral size = 16 bit
// memory increment enabled
// circular mode enabled
// memory to peripheral
DMA2_Channel16->CCR =
    (1 << 11) |
    (1 << 9) |
    (1 << 8) |
    (1 << 7) |
    (1 << 5) |
    (1 << 0);
}

void initClocks(void)
{
    // disable main PLL
    RCC->CR &= ~(1 << 24);
    while (RCC->CR & (1 << 25)) {}

    // MSI 4 MHz to 80 MHz system clock
    RCC->PLLCFGR =
        (1 << 25) |
        (1 << 24) |
        (1 << 22) |
        (1 << 21) |
        (1 << 17) |
        (80 << 8) |
        (1 << 0);

    // enable PLL
    RCC->CR |= (1 << 24);
    while ((RCC->CR & (1 << 25)) == 0) {}

    // flash wait states for 80 MHz
    FLASH->ACR &= ~0x7;
    FLASH->ACR |= 4;

    // select PLL as system clock
    RCC->CFGR &= ~0x3;
    RCC->CFGR |= 0x3;

    while ((RCC->CFGR & (0x3 << 2)) != (0x3 << 2)) {}

    SystemCoreClock = 80000000;
}

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)

```

```

{
    Port->MODER &= ~(3U << (BitNumber * 2));
    Port->MODER |= (Mode << (BitNumber * 2));
}

void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF)
{
    if (BitNumber < 8)
    {
        Port->AFR[0] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[0] |= (AF << (4 * BitNumber));
    }
    else
    {
        BitNumber -= 8;
        Port->AFR[1] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[1] |= (AF << (4 * BitNumber));
    }
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

```

Result: Audio output was produced through the speaker, verifying that the SAI peripheral, DMA transfer and MAX98357A amplifier were operating. However, some buffers produced noise or unclear playback, showing that correct I2S formatting, stereo sample duplication, DMA length and sample amplitude are critical for reliable audio playback.

Task 4: Timer-Based Audio Control

Goal: The goal of this task was to introduce timer-based behaviour into the embedded audio system. A hardware timer interrupt was configured to generate periodic events at fixed intervals. This moved the system away from relying entirely on polling inside the main loop and allowed time-critical events to occur independently of processor execution timing.

A timer interrupt was used to periodically toggle a heartbeat GPIO output, providing a visible indication that interrupt-driven timing was functioning correctly. The timer could also be extended to support synchronised audio events, playback timing, or periodic control updates in later stages of the project.

Using interrupts is important in embedded systems because it allows the processor to perform other tasks while timing operations continue accurately in the background. This approach is more scalable and efficient than continuously polling delays inside the main loop.

```
#include "stm32l432xx.h"
#include <stdint.h>
#include "audio_data.h"

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF);
void initClocks(void);
void initSAI(void);
void initDMA(void);
void initTIM2(void);

volatile uint32_t heartbeat = 0;

int main(void)
{
    initClocks();

    // enable GPIOA and GPIOB
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);

    // PB3 output
    pinMode(GPIOB, 3, 1);

    initSAI();

    initDMA();

    initTIM2();

    // enable SAI
    SAI1_Block_A->CR1 |= (1 << 16);

    while (1)
    {
        // main loop now mostly idle
        // timer interrupt handles periodic events
    }
}

void initTIM2(void)
{
    // enable TIM2 clock
```

```

RCC->APB1ENR1 |= (1 << 0);

/*
    timer clock = 80 MHz

    prescaler = 8000-1
    timer freq = 10 kHz

    auto reload = 10000-1
    interrupt every 1 second
*/

TIM2->PSC = 8000 - 1;

TIM2->ARR = 10000 - 1;

// enable update interrupt
TIM2->DIER |= (1 << 0);

// enable TIM2 interrupt in NVIC
NVIC_EnableIRQ(TIM2_IRQn);

// start timer
TIM2->CR1 |= (1 << 0);
}

void TIM2_IRQHandler(void)
{
    // check update interrupt flag
    if (TIM2->SR & (1 << 0))
    {
        // clear interrupt flag
        TIM2->SR &= ~(1 << 0);

        // toggle heartbeat LED
        GPIOB->ODR ^= (1 << 3);

        heartbeat++;
    }
}

void initSAI(void)
{
    // PA8 = BCLK
    // PA9 = LRC
    // PA10 = DIN

    pinMode(GPIOA, 8, 2);

```

```

pinMode(GPIOA, 9, 2);
pinMode(GPIOA, 10, 2);

selectAlternateFunction(GPIOA, 8, 13);
selectAlternateFunction(GPIOA, 9, 13);
selectAlternateFunction(GPIOA, 10, 13);

RCC->APB2ENR |= (1 << 21);

RCC->CR &= ~(1 << 26);
while (RCC->CR & (1 << 27)) {}

RCC->PLLSAI1CFGR =
    (2 << 27) |
    (37 << 8) |
    (1 << 16);

RCC->CR |= (1 << 26);
while ((RCC->CR & (1 << 27)) == 0) {}

SAI1->GCR = (1 << 4);

SAI1_Block_A->CR1 = 0;
SAI1_Block_A->CR2 = 0;
SAI1_Block_A->SLOTR = 0;
SAI1_Block_A->FRCR = 0;

SAI1_Block_A->CR1 =
    (7 << 5) |
    (3 << 20) |
    (1 << 9);

SAI1_Block_A->SLOTR =
    (1 << 8) |
    (1 << 16) |
    (1 << 17) |
    (1 << 7);

SAI1_Block_A->FRCR =
    (1 << 18) |
    (1 << 16) |
    (15 << 8) |
    (32 - 1);

// DMA enable
SAI1_Block_A->CR1 |= (1 << 17);
}

```



```

void initDMA(void)
{
    RCC->AHB1ENR |= (1 << 1);

    DMA2_Channel6->CCR &= ~(1 << 0);

    DMA2_Channel6->CMAR = (uint32_t)audio_data;

    DMA2_Channel6->CPAR = (uint32_t)&SAI1_Block_A->DR;

    DMA2_Channel6->CNDTR = AUDIO_LENGTH;

    DMA2_CSELR->CSELR &= ~(0xF << 20);
    DMA2_CSELR->CSELR |= (5 << 20);

    DMA2_Channel6->CCR =
        (1 << 11) |
        (1 << 9) |
        (1 << 8) |
        (1 << 7) |
        (1 << 5) |
        (1 << 0);
}

```

```

void initClocks(void)
{
    RCC->CR &= ~(1 << 24);

    while (RCC->CR & (1 << 25)) {}

    RCC->PLLCFGR =
        (1 << 25) |
        (1 << 24) |
        (1 << 22) |
        (1 << 21) |
        (1 << 17) |
        (80 << 8) |
        (1 << 0);

    RCC->CR |= (1 << 24);

    while ((RCC->CR & (1 << 25)) == 0) {}

    FLASH->ACR &= ~0x7;
    FLASH->ACR |= 4;

    RCC->CFGR &= ~0x3;
    RCC->CFGR |= 0x3;
}

```

```

while ((RCC->CFGR & (0x3 << 2)) != (0x3 << 2)) {}

SystemCoreClock = 80000000;
}

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    Port->MODER &= ~(3U << (BitNumber * 2));

    Port->MODER |= (Mode << (BitNumber * 2));
}

void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t
AF)
{
    if (BitNumber < 8)
    {
        Port->AFR[0] &= ~(0xFU << (4 * BitNumber));

        Port->AFR[0] |= (AF << (4 * BitNumber));
    }
    else
    {
        BitNumber -= 8;

        Port->AFR[1] &= ~(0xFU << (4 * BitNumber));

        Port->AFR[1] |= (AF << (4 * BitNumber));
    }
}

```

Results: TIM2 was configured to generate a periodic interrupt. Instead of using a delay inside the main loop, the heartbeat GPIO on PB3 was toggled inside the TIM2_IRQHandler() interrupt service routine. This verified that the STM32 was responding to timer interrupts correctly.

The audio playback continued to be handled separately by SAI and DMA, while the timer interrupt controlled periodic system behaviour in the background. This showed that the processor could manage timed events without constantly polling in the main loop.

Task 5: DMA-Based ADC Sampling

The goal of this task was to implement continuous ADC sampling using DMA while introducing an analogue user input through a potentiometer connected to the STM32. A potentiometer was added with its output connected to PA0, allowing changing analogue voltage levels to be sampled continuously by ADC1.

DMA was used in circular mode so that ADC samples were transferred directly into memory without requiring the CPU to manually poll the ADC peripheral. A buffer averaging method was implemented to reduce noise and produce smoother ADC readings. Timer interrupts were then used to periodically calculate the average ADC value and output it over UART.

potentiometer side 1 -> 3V3

potentiometer side 2 -> GND

potentiometer middle -> PA0

```
#include "stm32l432xx.h"
#include <stdint.h>
#include <stdio.h>
#include <sys/unistd.h>

#include "audio_data.h"

#define ADC_BUF_SIZE 32

volatile uint16_t adc_buffer[ADC_BUF_SIZE];
volatile uint16_t adc_average = 0;
volatile uint8_t print_flag = 0;

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF);

void initClocks(void);
void initUART(void);
void initSAI(void);
void initAudioDMA(void);
void initADC_DMA(void);
void initTIM2(void);
void delay(volatile uint32_t dly);

int _write(int file, char *ptr, int len);
```

```

int main(void)
{
    initClocks();

    RCC->AHB2ENR |= (1 << 0) | (1 << 1); // GPIOA and GPIOB

    // PB3 heartbeat
    pinMode(GPIOB, 3, 1);

    initUART();

    initSAI();
    initAudioDMA();

    initADC_DMA();

    initTIM2();

    // start SAI after DMA is ready
    SAI1_Block_A->CR1 |= (1 << 16);

    printf("Task 5 ADC DMA started\r\n");

    while (1)
    {
        if (print_flag)
        {
            print_flag = 0;
            printf("%d\r\n", adc_average);
        }

        // sleep until interrupt
        __WFI();
    }
}

void initADC_DMA(void)
{
    /*
        ADC input:
        PA0 = ADC1_IN5
    */

    // PA0 analog mode
    pinMode(GPIOA, 0, 3);

    // enable ADC clock

```

```

RCC->AHB2ENR |= (1 << 13);

// enable DMA1 clock
RCC->AHB1ENR |= (1 << 0);

// disable ADC before config
if (ADC1->CR & ADC_CR_ADEN)
{
    ADC1->CR |= ADC_CR_ADDIS;
    while (ADC1->CR & ADC_CR_ADEN) {}
}

// take ADC out of deep power down
ADC1->CR &= ~ADC_CR_DEEPPWD;

// enable ADC voltage regulator
ADC1->CR |= ADC_CR_ADVREGEN;
delay(10000);

// calibrate ADC
ADC1->CR |= ADC_CR_ADCAL;
while (ADC1->CR & ADC_CR_ADCAL) {}

// DMA1 channel 1 for ADC1
DMA1_Channel1->CCR &= ~(1 << 0);

DMA1_Channel1->CPAR = (uint32_t)&ADC1->DR;
DMA1_Channel1->CMAR = (uint32_t)adc_buffer;
DMA1_Channel1->CNDTR = ADC_BUF_SIZE;

// request select for ADC1 on DMA1 channel 1
DMA1_CSELR->CSELR &= ~(0xF << 0);

// DMA config:
// peripheral to memory
// memory increment
// circular mode
// peripheral size 16 bit
// memory size 16 bit
DMA1_Channel1->CCR =
    (1 << 10) | // memory size 16 bit
    (1 << 8) | // peripheral size 16 bit
    (1 << 7) | // circular mode
    (1 << 5); // memory increment

// enable DMA
DMA1_Channel1->CCR |= (1 << 0);

```

```

// ADC continuous conversion + DMA circular
ADC1->CFGR = 0;
ADC1->CFGR |= ADC_CFGR_CONT;
ADC1->CFGR |= ADC_CFGR_DMAEN;
ADC1->CFGR |= ADC_CFGR_DMACFG;

// sample time for channel 5
ADC1->SMPR1 &= ~(7 << (5 * 3));
ADC1->SMPR1 |= (7 << (5 * 3));

// one conversion, channel 5
ADC1->SQR1 = 0;
ADC1->SQR1 |= (5 << 6);

// enable ADC
ADC1->CR |= ADC_CR_ADEN;
while (!(ADC1->ISR & ADC_ISR_ADRDY)) {}

// start continuous conversion
ADC1->CR |= ADC_CR_ADSTART;
}

void initTIM2(void)
{
    // enable TIM2 clock
    RCC->APB1ENR1 |= (1 << 0);

    /*
        80 MHz clock
        PSC = 8000 - 1 gives 10 kHz timer count
        ARR = 1000 - 1 gives interrupt every 0.1 s
    */

    TIM2->PSC = 8000 - 1;
    TIM2->ARR = 1000 - 1;

    // enable update interrupt
    TIM2->DIER |= (1 << 0);

    NVIC_EnableIRQ(TIM2_IRQn);

    // start timer
    TIM2->CR1 |= (1 << 0);
}

void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1 << 0))

```

```

{
    TIM2->SR &= ~(1 << 0);

    uint32_t sum = 0;

    for (int i = 0; i < ADC_BUF_SIZE; i++)
    {
        sum += adc_buffer[i];
    }

    adc_average = sum / ADC_BUF_SIZE;

    // heartbeat
    GPIOB->ODR ^= (1 << 3);

    // tell main loop to print
    print_flag = 1;
}
}

void initUART(void)
{
    /*
        USART2 TX on PA2
        Open serial monitor at 115200 baud
    */

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->BRR = 8000000 / 115200;

    USART2->CR1 |= (1 << 3); // TX enable
    USART2->CR1 |= (1 << 0); // USART enable
}

int _write(int file, char *ptr, int len)
{
    for (int i = 0; i < len; i++)
    {
        while (!(USART2->ISR & (1 << 7))) {}
        USART2->TDR = ptr[i];
    }

    return len;
}

```

```

void initSAI(void)
{
    // PA8  = BCLK
    // PA9  = LRC
    // PA10 = DIN

    pinMode(GPIOA, 8, 2);
    pinMode(GPIOA, 9, 2);
    pinMode(GPIOA, 10, 2);

    selectAlternateFunction(GPIOA, 8, 13);
    selectAlternateFunction(GPIOA, 9, 13);
    selectAlternateFunction(GPIOA, 10, 13);

    RCC->APB2ENR |= (1 << 21);

    RCC->CR &= ~(1 << 26);
    while (RCC->CR & (1 << 27)) {}

    RCC->PLLSAI1CFGR =
        (2 << 27) |
        (37 << 8) |
        (1 << 16);

    RCC->CR |= (1 << 26);
    while ((RCC->CR & (1 << 27)) == 0) {}

    SAI1->GCR = (1 << 4);

    SAI1_Block_A->CR1 = 0;
    SAI1_Block_A->CR2 = 0;
    SAI1_Block_A->SLOTR = 0;
    SAI1_Block_A->FRCR = 0;

    SAI1_Block_A->CR1 =
        (7 << 5) |
        (3 << 20) |
        (1 << 9);

    SAI1_Block_A->SLOTR =
        (1 << 8) |
        (1 << 16) |
        (1 << 17) |
        (1 << 7);

    SAI1_Block_A->FRCR =
        (1 << 18) |

```



```

        (1 << 16) |
        (15 << 8) |
        (32 - 1);

    SAI1_Block_A->CR1 |= (1 << 17);
}

void initAudioDMA(void)
{
    // DMA2 channel 6 for SAI audio
    RCC->AHB1ENR |= (1 << 1);

    DMA2_Channel6->CCR &= ~(1 << 0);

    DMA2_Channel6->CMAR = (uint32_t)audio_data;
    DMA2_Channel6->CPAR = (uint32_t)&SAI1_Block_A->DR;
    DMA2_Channel6->CNDTR = AUDIO_LENGTH;

    DMA2_CSELR->CSELR &= ~(0xF << 20);
    DMA2_CSELR->CSELR |= (5 << 20);

    DMA2_Channel6->CCR =
        (1 << 11) |
        (1 << 9) |
        (1 << 8) |
        (1 << 7) |
        (1 << 5) |
        (1 << 0);
}

void initClocks(void)
{
    RCC->CR &= ~(1 << 24);
    while (RCC->CR & (1 << 25)) {}

    RCC->PLLCFGR =
        (1 << 25) |
        (1 << 24) |
        (1 << 22) |
        (1 << 21) |
        (1 << 17) |
        (80 << 8) |
        (1 << 0);

    RCC->CR |= (1 << 24);
    while ((RCC->CR & (1 << 25)) == 0) {}

    FLASH->ACR &= ~0x7;

```

```

FLASH->ACR |= 4;

RCC->CFGR &= ~0x3;
RCC->CFGR |= 0x3;

while ((RCC->CFGR & (0x3 << 2)) != (0x3 << 2)) {}

SystemCoreClock = 80000000;
}

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    Port->MODER &= ~(3U << (BitNumber * 2));
    Port->MODER |= (Mode << (BitNumber * 2));
}

void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t
AF)
{
    if (BitNumber < 8)
    {
        Port->AFR[0] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[0] |= (AF << (4 * BitNumber));
    }
    else
    {
        BitNumber -= 8;
        Port->AFR[1] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[1] |= (AF << (4 * BitNumber));
    }
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

```

Result: The ADC samples continuously into adc_buffer[] using DMA. The CPU does not manually read the ADC. TIM2 periodically averages the buffer and prints the value over UART. Turning the potesntiometer should smoothly change the printed ADC value from near 0 to around 4095.

Task 6: Audio Response Based on Tilt Direction

Goal: Generate digital sine wave audio samples in MATLAB and use them as a lookup table for real-time audio playback on the STM32L432 using SAI/I²S and DMA. The objective was to verify successful serial audio transmission and continuous waveform playback through the speaker module.

```
#include "stm32l432xx.h"
#include <stdint.h>
#include <stdio.h>
#include <sys/unistd.h>

#include "audio_data.h"

#define ADC_BUF_SIZE 32

volatile uint16_t adc_buffer[ADC_BUF_SIZE];
volatile uint16_t adc_average = 0;
volatile uint8_t print_flag = 0;

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF);

void initClocks(void);
void initUART(void);
void initSAI(void);
void initAudioDMA(void);
void initADC_DMA(void);
void initTIM2(void);
void delay(volatile uint32_t dly);

int _write(int file, char *ptr, int len);

int main(void)
{
    initClocks();

    RCC->AHB2ENR |= (1 << 0) | (1 << 1);

    pinMode(GPIOB, 3, 1);    // heartbeat/debug pin

    initUART();

    printf("Project 2 audio and ADC DMA test\r\n");
```

```

initSAI();
initAudioDMA();

initADC_DMA();
initTIM2();

// enable SAI block A after DMA is ready
SAI1_Block_A->CR1 |= (1 << 16);

printf("SAI audio DMA started\r\n");
printf("ADC DMA started\r\n");

while (1)
{
    if (print_flag)
    {
        print_flag = 0;
        printf("ADC average = %d\r\n", adc_average);
    }

    __WFI();
}
}

void initADC_DMA(void)
{
    // PA0 = ADC1_IN5
    pinMode(GPIOA, 0, 3);

    RCC->AHB2ENR |= (1 << 13); // ADC clock
    RCC->AHB1ENR |= (1 << 0); // DMA1 clock

    if (ADC1->CR & ADC_CR_ADEN)
    {
        ADC1->CR |= ADC_CR_ADDIS;
        while (ADC1->CR & ADC_CR_ADEN) {}
    }

    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    delay(10000);

    ADC1->CR |= ADC_CR_ADCAL;
    while (ADC1->CR & ADC_CR_ADCAL) {}

    DMA1_Channel1->CCR &= ~(1 << 0);

    DMA1_Channel1->CPAR = (uint32_t)&ADC1->DR;

```

```

DMA1_Channel1->CMAR = (uint32_t)adc_buffer;
DMA1_Channel1->CNDTR = ADC_BUF_SIZE;

DMA1_CSELR->CSELR &= ~(0xF << 0);

DMA1_Channel1->CCR =
    (1 << 10) |    // memory size 16 bit
    (1 << 8)  |    // peripheral size 16 bit
    (1 << 7)  |    // circular mode
    (1 << 5);     // memory increment

DMA1_Channel1->CCR |= (1 << 0);

ADC1->CFGR = 0;
ADC1->CFGR |= ADC_CFGR_CONT;
ADC1->CFGR |= ADC_CFGR_DMAEN;
ADC1->CFGR |= ADC_CFGR_DMACFG;

ADC1->SMPR1 &= ~(7 << (5 * 3));
ADC1->SMPR1 |= (7 << (5 * 3));

ADC1->SQR1 = 0;
ADC1->SQR1 |= (5 << 6);

ADC1->CR |= ADC_CR_ADEN;
while (!(ADC1->ISR & ADC_ISR_ADRDY)) {}

ADC1->CR |= ADC_CR_ADSTART;
}

void initTIM2(void)
{
    RCC->APB1ENR1 |= (1 << 0);

    TIM2->PSC = 8000 - 1;
    TIM2->ARR = 1000 - 1;

    TIM2->DIER |= (1 << 0);

    NVIC_EnableIRQ(TIM2_IRQn);

    TIM2->CR1 |= (1 << 0);
}

void TIM2_IRQHandler(void)
{
    if (TIM2->SR & (1 << 0))
    {

```

```

    TIM2->SR &= ~(1 << 0);

    uint32_t sum = 0;

    for (int i = 0; i < ADC_BUF_SIZE; i++)
    {
        sum += adc_buffer[i];
    }

    adc_average = sum / ADC_BUF_SIZE;

    GPIOB->ODR ^= (1 << 3);

    print_flag = 1;
}
}

void initUART(void)
{
    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->BRR = 8000000 / 115200;

    USART2->CR1 = 0;
    USART2->CR1 |= (1 << 3);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *ptr, int len)
{
    for (int i = 0; i < len; i++)
    {
        while (!(USART2->ISR & (1 << 7))) {}
        USART2->TDR = ptr[i];
    }

    return len;
}

void initSAI(void)
{
    // PA8 = BCLK
    // PA9 = LRC
    // PA10 = DIN

```

```

pinMode(GPIOA, 8, 2);
pinMode(GPIOA, 9, 2);
pinMode(GPIOA, 10, 2);

selectAlternateFunction(GPIOA, 8, 13);
selectAlternateFunction(GPIOA, 9, 13);
selectAlternateFunction(GPIOA, 10, 13);

RCC->APB2ENR |= (1 << 21);

RCC->CR &= ~(1 << 26);
while (RCC->CR & (1 << 27)) {}

RCC->PLLSAI1CFGR =
    (2 << 27) |
    (37 << 8) |
    (1 << 16);

RCC->CR |= (1 << 26);
while ((RCC->CR & (1 << 27)) == 0) {}

SAI1->GCR = (1 << 4);

SAI1_Block_A->CR1 = 0;
SAI1_Block_A->CR2 = 0;
SAI1_Block_A->SLOTR = 0;
SAI1_Block_A->FRCR = 0;

SAI1_Block_A->CR1 =
    (7 << 5) |
    (3 << 20) |
    (1 << 9);

SAI1_Block_A->SLOTR =
    (1 << 8) |
    (1 << 16) |
    (1 << 17) |
    (1 << 7);

SAI1_Block_A->FRCR =
    (1 << 18) |
    (1 << 16) |
    (15 << 8) |
    (32 - 1);

SAI1_Block_A->CR1 |= (1 << 17);
}

```

```

void initAudioDMA(void)
{
    RCC->AHB1ENR |= (1 << 1);

    DMA2_Channel16->CCR &= ~(1 << 0);

    DMA2_Channel16->CMAR = (uint32_t)audio_data;
    DMA2_Channel16->CPAR = (uint32_t)&SAI1_Block_A->DR;
    DMA2_Channel16->CNDTR = AUDIO_LENGTH;

    DMA2_CSELR->CSELR &= ~(0xF << 20);

    // If audio does not play, try changing this to (1 << 20)
    DMA2_CSELR->CSELR |= (5 << 20);

    DMA2_Channel16->CCR =
        (1 << 11) |
        (1 << 9) |
        (1 << 8) |
        (1 << 7) |
        (1 << 5) |
        (1 << 0);
}

void initClocks(void)
{
    RCC->CR &= ~(1 << 24);
    while (RCC->CR & (1 << 25)) {}

    RCC->PLLCFGR =
        (1 << 25) |
        (1 << 24) |
        (1 << 22) |
        (1 << 21) |
        (1 << 17) |
        (80 << 8) |
        (1 << 0);

    RCC->CR |= (1 << 24);
    while ((RCC->CR & (1 << 25)) == 0) {}

    FLASH->ACR &= ~0x7;
    FLASH->ACR |= 4;

    RCC->CFGR &= ~0x3;
    RCC->CFGR |= 0x3;

    while ((RCC->CFGR & (0x3 << 2)) != (0x3 << 2)) {}
}

```



```

        SystemCoreClock = 80000000;
    }

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    Port->MODER &= ~(3U << (BitNumber * 2));
    Port->MODER |= (Mode << (BitNumber * 2));
}

void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t
AF)
{
    if (BitNumber < 8)
    {
        Port->AFR[0] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[0] |= (AF << (4 * BitNumber));
    }
    else
    {
        BitNumber -= 8;
        Port->AFR[1] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[1] |= (AF << (4 * BitNumber));
    }
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

```

Audio_data.h file

```

#ifndef AUDIO_DATA_H
#define AUDIO_DATA_H

#include <stdint.h>

#define AUDIO_LENGTH 161

const int16_t audio_data[AUDIO_LENGTH] = {
    0,
    3851,
    7649,
    11341,
    14876,
    18204,
    21280,

```

24062,
26509,
28589,
30273,
31537,
32364,
32742,
32666,
32137,
31163,
29757,
27938,
25732,
23170,
20286,
17121,
13718,
10126,
6393,
2571,
-1286,
-5126,
-8894,
-12539,
-16011,
-19260,
-22242,
-24916,
-27245,
-29196,
-30742,
-31862,
-32540,
-32767,
-32540,
-31862,
-30742,
-29196,
-27245,
-24916,
-22242,
-19260,
-16011,
-12539,
-8894,
-5126,
-1286,
2571,

6393,
10126,
13718,
17121,
20286,
23170,
25732,
27938,
29757,
31163,
32137,
32666,
32742,
32364,
31537,
30273,
28589,
26509,
24062,
21280,
18204,
14876,
11341,
7649,
3851,
0,
-3851,
-7649,
-11341,
-14876,
-18204,
-21280,
-24062,
-26509,
-28589,
-30273,
-31537,
-32364,
-32742,
-32666,
-32137,
-31163,
-29757,
-27938,
-25732,
-23170,
-20286,
-17121,

-13718,
-10126,
-6393,
-2571,
1286,
5126,
8894,
12539,
16011,
19260,
22242,
24916,
27245,
29196,
30742,
31862,
32540,
32767,
32540,
31862,
30742,
29196,
27245,
24916,
22242,
19260,
16011,
12539,
8894,
5126,
1286,
-2571,
-6393,
-10126,
-13718,
-17121,
-20286,
-23170,
-25732,
-27938,
-29757,
-31163,
-32137,
-32666,
-32742,
-32364,
-31537,
-30273,

```
-28589,  
-26509,  
-24062,  
-21280,  
-18204,  
-14876,  
-11341,  
-7649,  
-3851,  
0  
};  
  
#endif
```

Results

MATLAB was used to generate 16-bit sine wave sample arrays for multiple frequencies including 300 Hz, 600 Hz, 900 Hz, and 1200 Hz. The generated samples were exported into a C header file (audio_data.h) and stored as an integer lookup table.

The STM32 successfully transmitted the waveform data using the SAI peripheral configured in I²S mode. DMA channel 6 continuously transferred audio samples from memory to the SAI data register using circular mode operation.

A continuous 300 Hz tone was successfully heard from the speaker, confirming:

- correct SAI/I²S configuration
- successful DMA audio streaming
- correct clock generation
- valid digital audio sample generation
- successful integration of MATLAB-generated waveform data into embedded C code

The system was also able to simultaneously:

- perform ADC sampling using DMA
- execute timer interrupts
- print UART debug information
- maintain continuous audio playback

Goal

Use the BMI160 accelerometer readings to control the I²S speaker output so that each tilt direction produces a different tone.

```
#include "stm32l432xx.h"
#include <stdint.h>
#include <stdio.h>
#include <sys/unistd.h>

#include "audio_data.h"
#include "i2c.h"
#include "eeng1030_lib.h"

#define ADC_BUF_SIZE 32

#define LEFT_THRESHOLD      600
#define RIGHT_THRESHOLD    -600
#define FORWARD_THRESHOLD  600
#define BACKWARD_THRESHOLD -600

volatile uint16_t adc_buffer[ADC_BUF_SIZE];
volatile uint16_t adc_average = 0;
volatile uint8_t timer_flag = 0;

uint16_t response;
int32_t X_g = 0;
int32_t Y_g = 0;

const int16_t *current_tone = 0;

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode);
void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t AF);

void initClocks(void);
void initUART(void);
void initSAI(void);
void initAudioDMA(const int16_t *tone);
void initADC_DMA(void);
void initTIM2(void);
void delay(volatile uint32_t dly);

void bmi160_init(void);
int32_t read_bmi160_axis(uint8_t reg_low);

void set_audio_tone(const int16_t *tone);
void update_audio_from_tilt(void);
```

```

int _write(int file, char *ptr, int len);

int main(void)
{
    initClocks();

    RCC->AHB2ENR |= (1 << 0) | (1 << 1);

    pinMode(GPIOB, 3, 1);    // heartbeat/debug pin

    initUART();

    printf("Project 2 tilt controlled I2S audio\r\n");

    initI2C();
    ResetI2C();
    bmi160_init();

    initSAI();
    initAudioDMA(tone_silent);

    initADC_DMA();
    initTIM2();

    SAI1_Block_A->CR1 |= (1 << 16);

    printf("SAI DMA audio started\r\n");
    printf("ADC DMA started\r\n");
    printf("Tilt controlled tones active\r\n");

    while (1)
    {
        if (timer_flag)
        {
            timer_flag = 0;

            X_g = read_bmi160_axis(0x12);
            Y_g = read_bmi160_axis(0x14);

            update_audio_from_tilt();

            printf("X=%ld Y=%ld ADC=%d\r\n", X_g, Y_g, adc_average);
        }

        __WFI();
    }
}

```

```

void update_audio_from_tilt(void)
{
    if (X_g > LEFT_THRESHOLD)
    {
        set_audio_tone(tone_300Hz);
    }
    else if (X_g < RIGHT_THRESHOLD)
    {
        set_audio_tone(tone_600Hz);
    }
    else if (Y_g > FORWARD_THRESHOLD)
    {
        set_audio_tone(tone_900Hz);
    }
    else if (Y_g < BACKWARD_THRESHOLD)
    {
        set_audio_tone(tone_1200Hz);
    }
    else
    {
        set_audio_tone(tone_silent);
    }
}

void set_audio_tone(const int16_t *tone)
{
    if (current_tone == tone)
    {
        return;
    }

    current_tone = tone;

    DMA2_Channel16->CCR &= ~(1 << 0);
    SAI1_Block_A->CR1 &= ~(1 << 16);

    DMA2_Channel16->CMAR = (uint32_t)tone;
    DMA2_Channel16->CNDTR = AUDIO_LENGTH;

    DMA2_Channel16->CCR |= (1 << 0);
    SAI1_Block_A->CR1 |= (1 << 16);
}

void bmi160_init(void)
{
    I2CStart(0x69, WRITE, 2);
    I2CWrite(0x7E);
    I2CWrite(0x11);
}

```



```

    I2CStop();

    delay(1000000);
}

int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

    I2CStart(0x69, WRITE, 1);
    I2CWrite(reg_low);
    I2CReStart(0x69, READ, 2);

    response = I2CRead();
    accel_raw = response;

    response = I2CRead();
    accel_raw = accel_raw + (response << 8);

    I2CStop();

    value = accel_raw;
    value = (value * 981) / 16384;

    return value;
}

void initADC_DMA(void)
{
    pinMode(GPIOA, 0, 3);

    RCC->AHB2ENR |= (1 << 13);
    RCC->AHB1ENR |= (1 << 0);

    if (ADC1->CR & ADC_CR_ADEN)
    {
        ADC1->CR |= ADC_CR_ADDIS;
        while (ADC1->CR & ADC_CR_ADEN) {}
    }

    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    delay(10000);

    ADC1->CR |= ADC_CR_ADCAL;
    while (ADC1->CR & ADC_CR_ADCAL) {}
}

```

```

DMA1_Channel1->CCR &= ~(1 << 0);

DMA1_Channel1->CPAR = (uint32_t)&ADC1->DR;
DMA1_Channel1->CMAR = (uint32_t)adc_buffer;
DMA1_Channel1->CNDTR = ADC_BUF_SIZE;

DMA1_CSELR->CSELR &= ~(0xF << 0);

DMA1_Channel1->CCR =
    (1 << 10) |
    (1 << 8) |
    (1 << 7) |
    (1 << 5);

DMA1_Channel1->CCR |= (1 << 0);

ADC1->CFGR = 0;
ADC1->CFGR |= ADC_CFGR_CONT;
ADC1->CFGR |= ADC_CFGR_DMAEN;
ADC1->CFGR |= ADC_CFGR_DMACFG;

ADC1->SMPR1 &= ~(7 << (5 * 3));
ADC1->SMPR1 |= (7 << (5 * 3));

ADC1->SQR1 = 0;
ADC1->SQR1 |= (5 << 6);

ADC1->CR |= ADC_CR_ADEN;
while (!(ADC1->ISR & ADC_ISR_ADRDY)) {}

ADC1->CR |= ADC_CR_ADSTART;
}

void initTIM2(void)
{
    RCC->APB1ENR1 |= (1 << 0);

    TIM2->PSC = 8000 - 1;
    TIM2->ARR = 1000 - 1;

    TIM2->DIER |= (1 << 0);

    NVIC_EnableIRQ(TIM2_IRQn);

    TIM2->CR1 |= (1 << 0);
}

void TIM2_IRQHandler(void)

```

```

{
    if (TIM2->SR & (1 << 0))
    {
        TIM2->SR &= ~(1 << 0);

        uint32_t sum = 0;

        for (int i = 0; i < ADC_BUF_SIZE; i++)
        {
            sum += adc_buffer[i];
        }

        adc_average = sum / ADC_BUF_SIZE;

        GPIOB->ODR ^= (1 << 3);

        timer_flag = 1;
    }
}

void initUART(void)
{
    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->BRR = 80000000 / 115200;

    USART2->CR1 = 0;
    USART2->CR1 |= (1 << 3);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *ptr, int len)
{
    for (int i = 0; i < len; i++)
    {
        while (!(USART2->ISR & (1 << 7))) {}
        USART2->TDR = ptr[i];
    }

    return len;
}

void initSAI(void)
{
    pinMode(GPIOA, 8, 2);

```

```

pinMode(GPIOA, 9, 2);
pinMode(GPIOA, 10, 2);

selectAlternateFunction(GPIOA, 8, 13);
selectAlternateFunction(GPIOA, 9, 13);
selectAlternateFunction(GPIOA, 10, 13);

RCC->APB2ENR |= (1 << 21);

RCC->CR &= ~(1 << 26);
while (RCC->CR & (1 << 27)) {}

RCC->PLLSAI1CFGR =
    (2 << 27) |
    (37 << 8) |
    (1 << 16);

RCC->CR |= (1 << 26);
while ((RCC->CR & (1 << 27)) == 0) {}

SAI1->GCR = (1 << 4);

SAI1_Block_A->CR1 = 0;
SAI1_Block_A->CR2 = 0;
SAI1_Block_A->SLOTR = 0;
SAI1_Block_A->FRCR = 0;

SAI1_Block_A->CR1 =
    (7 << 5) |
    (3 << 20) |
    (1 << 9);

SAI1_Block_A->SLOTR =
    (1 << 8) |
    (1 << 16) |
    (1 << 17) |
    (1 << 7);

SAI1_Block_A->FRCR =
    (1 << 18) |
    (1 << 16) |
    (15 << 8) |
    (32 - 1);

SAI1_Block_A->CR1 |= (1 << 17);
}

void initAudioDMA(const int16_t *tone)

```

```

{
    current_tone = tone;

    RCC->AHB1ENR |= (1 << 1);

    DMA2_Channel16->CCR &= ~(1 << 0);

    DMA2_Channel16->CMAR = (uint32_t)tone;
    DMA2_Channel16->CPAR = (uint32_t)&SAI1_Block_A->DR;
    DMA2_Channel16->CNDTR = AUDIO_LENGTH;

    DMA2_CSELR->CSELR &= ~(0xF << 20);
    DMA2_CSELR->CSELR |= (5 << 20);

    DMA2_Channel16->CCR =
        (1 << 11) |
        (1 << 9) |
        (1 << 8) |
        (1 << 7) |
        (1 << 5) |
        (1 << 0);
}

void initClocks(void)
{
    RCC->CR &= ~(1 << 24);
    while (RCC->CR & (1 << 25)) {}

    RCC->PLLCFGR =
        (1 << 25) |
        (1 << 24) |
        (1 << 22) |
        (1 << 21) |
        (1 << 17) |
        (80 << 8) |
        (1 << 0);

    RCC->CR |= (1 << 24);
    while ((RCC->CR & (1 << 25)) == 0) {}

    FLASH->ACR &= ~0x7;
    FLASH->ACR |= 4;

    RCC->CFGR &= ~0x3;
    RCC->CFGR |= 0x3;

    while ((RCC->CFGR & (0x3 << 2)) != (0x3 << 2)) {}
}

```

```

    SystemCoreClock = 80000000;
}

void pinMode(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t Mode)
{
    Port->MODER &= ~(3U << (BitNumber * 2));
    Port->MODER |= (Mode << (BitNumber * 2));
}

void selectAlternateFunction(GPIO_TypeDef *Port, uint32_t BitNumber, uint32_t
AF)
{
    if (BitNumber < 8)
    {
        Port->AFR[0] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[0] |= (AF << (4 * BitNumber));
    }
    else
    {
        BitNumber -= 8;
        Port->AFR[1] &= ~(0xFU << (4 * BitNumber));
        Port->AFR[1] |= (AF << (4 * BitNumber));
    }
}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

```

Result: The system successfully used the accelerometer X and Y values to select different audio tones. When the board was tilted left, right, forward, or backward, the software changed the active DMA audio buffer and played a different tone through the I²S speaker.

The audio output continued to run using DMA, while the timer interrupt controlled periodic updates and the UART printed the current X and Y tilt values. The flat position selected a silent buffer, muting the speaker when no clear tilt direction was detected.

This confirmed that the I²C accelerometer, timer interrupt, DMA audio transfer, and SAI/I²S speaker output were working together in one integrated embedded system.

Task 7: Final Integration, Optimisation and Testing

Goal: The goal of Task 7 was to integrate a 300 Hz audio warning into the final Project 1 tilt-controlled LED ring system while keeping the LED pointer, colour selection, saved-position feature, ADC input, PWM output, filtering, and UART debugging unchanged.

```
#include "stm32l4xx.h"
#include <stdint.h>
#include <stdio.h>
#include <errno.h>
#include <sys/unistd.h>

#include "eeng1030_lib.h"
#include "i2c.h"
#include "audio_data.h"

#define NUM_LEDS    16
#define WS_PIN      7

#define BTN_PIN     3          // button moved to PB3

#define TOP_LED     0
#define DEAD_ZONE   180
#define SAVE_THRESHOLD 200

#define AUDIO_LIMIT 750
#define ADC_BUF_SIZE 32

uint16_t response;

int32_t X_g, Y_g, Z_g;
int32_t X_filt = 0, Y_filt = 0, Z_filt = 0;

int saved_mode = 0;
int saved_centre_led = 0;
int save_latched = 0;

volatile uint16_t adc_buffer[ADC_BUF_SIZE];
volatile uint16_t adc_average = 0;
volatile uint8_t timer_flag = 0;

const int16_t tone_silent[AUDIO_LENGTH] = {0};
const int16_t *current_tone = 0;

static const int16_t dir_x[NUM_LEDS] = {
    0, 383, 707, 924, 1000, 924, 707, 383,
```

```

    0, -383, -707, -924, -1000, -924, -707, -383
};

static const int16_t dir_y[NUM_LEDS] = {
    1000,  924,  707,  383,    0, -383, -707, -924,
   -1000, -924, -707, -383,    0,  383,  707,  924
};

void setup(void);
void delay(volatile uint32_t dly);
void initSerial(uint32_t baudrate);
void eputc(char c);

static void clock_init_80mhz(void);
static void gpio_init_ws2812(void);
static void gpio_init_button(void);
static int button_pressed(void);
static void dwt_init(void);
static void delay_us(uint32_t us);

static void initADC_DMA(void);
static void initTimer2PWM(void);
static void setTimer2Duty(int duty);
static void initTIM6Interrupt(void);

static void initSAI(void);
static void initAudioDMA(const int16_t *tone);
static void set_audio_tone(const int16_t *tone);
static void update_audio_warning(void);

static void ws2812_reset(void);
static void ws2812_send_bit(uint8_t bit);
static void ws2812_send_byte(uint8_t byte);
static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b);

static int32_t read_bmi160_axis(uint8_t reg_low);
static int wrap_led(int n);
static int get_pointer_led(int32_t x, int32_t y, int32_t z);
static void show_live_and_saved(int live_centre_led, int saved_centre_led, int
saved_active,
                                uint8_t g, uint8_t r, uint8_t b);

int main(void)
{
    uint8_t colour_index = 0;
    uint8_t last_button_state = 0;

    uint8_t g_val = 0x00;

```



```

uint8_t r_val = 0x08;
uint8_t b_val = 0x00;

clock_init_80mhz();
setup();
initSerial(9600);

gpio_init_ws2812();
gpio_init_button();
dwt_init();

// PA3 = TIM2_CH4 PWM output
pinMode(GPIOA, 3, 2);
GPIOA->AFR[0] &= ~(0xF << (4 * 3));
GPIOA->AFR[0] |= (1U << (4 * 3));

initADC_DMA();
initTimer2PWM();

initSAI();
initAudioDMA(tone_silent);

initTIM6Interrupt();

ResetI2C();

I2CStart(0x69, WRITE, 2);
I2CWrite(0x7E);
I2CWrite(0x11);
I2CStop();

delay(1000000);

// start SAI after DMA is ready
SAI1_Block_A->CR1 |= (1 << 16);

printf("Project 2 final integration started\r\n");

while (1)
{
    if (timer_flag)
    {
        timer_flag = 0;
        printf("ADC average = %d\r\n", adc_average);
    }

    uint8_t current_button_state = button_pressed();

```

```

if ((current_button_state == 1) && (last_button_state == 0))
{
    delay(30000);

    if (button_pressed() == 1)
    {
        colour_index++;

        if (colour_index >= 4)
            colour_index = 0;

        while (button_pressed() == 1) {}

        delay(30000);
    }
}

last_button_state = current_button_state;

X_g = read_bmi160_axis(0x12);
Y_g = read_bmi160_axis(0x14);
Z_g = read_bmi160_axis(0x16);

X_filt = (3 * X_filt + X_g) / 4;
Y_filt = (3 * Y_filt + Y_g) / 4;
Z_filt = (3 * Z_filt + Z_g) / 4;

int pot_value = adc_average;
int centre_led = get_pointer_led(X_filt, Y_filt, Z_filt);

if (pot_value <= SAVE_THRESHOLD)
{
    setTimer2Duty(0);

    if (save_latched == 0)
    {
        saved_centre_led = centre_led;
        saved_mode = 1;
        save_latched = 1;
    }
}
else
{
    setTimer2Duty(pot_value);
    saved_mode = 0;
    save_latched = 0;
}

```

```

        if (colour_index == 0)
        {
            g_val = 0x00;
            r_val = 0x08;
            b_val = 0x00;
        }
        else if (colour_index == 1)
        {
            g_val = 0x08;
            r_val = 0x08;
            b_val = 0x00;
        }
        else if (colour_index == 2)
        {
            g_val = 0x02;
            r_val = 0x08;
            b_val = 0x00;
        }
        else
        {
            g_val = 0x08;
            r_val = 0x00;
            b_val = 0x00;
        }

        update_audio_warning();

        printf("POT=%d X=%ld Y=%ld Z=%ld colour=%d saved=%d saved_led=%d\n",
live_led=%d\r\n",
            pot_value, X_filt, Y_filt, Z_filt,
            colour_index, saved_mode, saved_centre_led, centre_led);

        show_live_and_saved(centre_led, saved_centre_led, saved_mode, g_val,
r_val, b_val);

        delay(80000);
    }
}

static void update_audio_warning(void)
{
    if (X_filt > AUDIO_LIMIT ||
        X_filt < -AUDIO_LIMIT ||
        Y_filt > AUDIO_LIMIT ||
        Y_filt < -AUDIO_LIMIT)
    {
        set_audio_tone(audio_data);
    }
}

```

```

    else
    {
        set_audio_tone(tone_silent);
    }
}

static void set_audio_tone(const int16_t *tone)
{
    if (current_tone == tone)
        return;

    current_tone = tone;

    DMA2_Channel16->CCR &= ~(1 << 0);
    SAI1_Block_A->CR1 &= ~(1 << 16);

    DMA2_Channel16->CMAR = (uint32_t)tone;
    DMA2_Channel16->CNDTR = AUDIO_LENGTH;

    DMA2_Channel16->CCR |= (1 << 0);
    SAI1_Block_A->CR1 |= (1 << 16);
}

static void initSAI(void)
{
    // PA8  = BCLK
    // PA9  = LRC
    // PA10 = DIN

    pinMode(GPIOA, 8, 2);
    pinMode(GPIOA, 9, 2);
    pinMode(GPIOA, 10, 2);

    selectAlternateFunction(GPIOA, 8, 13);
    selectAlternateFunction(GPIOA, 9, 13);
    selectAlternateFunction(GPIOA, 10, 13);

    RCC->APB2ENR |= (1 << 21);

    RCC->CR &= ~(1 << 26);
    while (RCC->CR & (1 << 27)) {}

    RCC->PLLSAI1CFGR =
        (2 << 27) |
        (37 << 8) |
        (1 << 16);

    RCC->CR |= (1 << 26);

```

```

while ((RCC->CR & (1 << 27)) == 0) {}

SAI1->GCR = (1 << 4);

SAI1_Block_A->CR1 = 0;
SAI1_Block_A->CR2 = 0;
SAI1_Block_A->SLOTR = 0;
SAI1_Block_A->FRCR = 0;

SAI1_Block_A->CR1 =
    (7 << 5) |
    (3 << 20) |
    (1 << 9);

SAI1_Block_A->SLOTR =
    (1 << 8) |
    (1 << 16) |
    (1 << 17) |
    (1 << 7);

SAI1_Block_A->FRCR =
    (1 << 18) |
    (1 << 16) |
    (15 << 8) |
    (32 - 1);

SAI1_Block_A->CR1 |= (1 << 17);
}

static void initAudioDMA(const int16_t *tone)
{
    current_tone = tone;

    RCC->AHB1ENR |= (1 << 1);

    DMA2_Channel6->CCR &= ~(1 << 0);

    DMA2_Channel6->CMAR = (uint32_t)tone;
    DMA2_Channel6->CPAR = (uint32_t)&SAI1_Block_A->DR;
    DMA2_Channel6->CNDTR = AUDIO_LENGTH;

    DMA2_CSELR->CSELR &= ~(0xF << 20);
    DMA2_CSELR->CSELR |= (5 << 20);

    DMA2_Channel6->CCR =
        (1 << 11) |
        (1 << 9) |
        (1 << 8) |

```

```

        (1 << 7) |
        (1 << 5) |
        (1 << 0);
}

static void initADC_DMA(void)
{
    // PA0 = ADC1_IN5 on STM32L432KC
    pinMode(GPIOA, 0, 3);

    RCC->AHB2ENR |= (1 << 13);
    RCC->AHB1ENR |= (1 << 0);

    if (ADC1->CR & ADC_CR_ADEN)
    {
        ADC1->CR |= ADC_CR_ADDIS;
        while (ADC1->CR & ADC_CR_ADEN) {}
    }

    ADC1->CR &= ~ADC_CR_DEEPPWD;
    ADC1->CR |= ADC_CR_ADVREGEN;
    delay(10000);

    ADC1->CR |= ADC_CR_ADCAL;
    while (ADC1->CR & ADC_CR_ADCAL) {}

    DMA1_Channel1->CCR &= ~(1 << 0);

    DMA1_Channel1->CPAR = (uint32_t)&ADC1->DR;
    DMA1_Channel1->CMAR = (uint32_t)adc_buffer;
    DMA1_Channel1->CNDTR = ADC_BUF_SIZE;

    DMA1_CSELR->CSELR &= ~(0xF << 0);

    DMA1_Channel1->CCR =
        (1 << 10) |
        (1 << 8) |
        (1 << 7) |
        (1 << 5);

    DMA1_Channel1->CCR |= (1 << 0);

    ADC1->CFGR = 0;
    ADC1->CFGR |= ADC_CFGR_CONT;
    ADC1->CFGR |= ADC_CFGR_DMAEN;
    ADC1->CFGR |= ADC_CFGR_DMACFG;

    ADC1->SMPR1 &= ~(7 << (5 * 3));

```

```

ADC1->SMPR1 |= (7 << (5 * 3));

ADC1->SQR1 = 0;
ADC1->SQR1 |= (5 << 6);

ADC1->CR |= ADC_CR_ADEN;
while (!(ADC1->ISR & ADC_ISR_ADRDY)) {}

ADC1->CR |= ADC_CR_ADSTART;
}

static void initTIM6Interrupt(void)
{
    RCC->APB1ENR1 |= (1 << 4);

    TIM6->PSC = 8000 - 1;
    TIM6->ARR = 1000 - 1;

    TIM6->DIER |= (1 << 0);

    NVIC_EnableIRQ(TIM6_DAC_IRQn);

    TIM6->CR1 |= (1 << 0);
}

void TIM6_DAC_IRQHandler(void)
{
    if (TIM6->SR & (1 << 0))
    {
        TIM6->SR &= ~(1 << 0);

        uint32_t sum = 0;

        for (int i = 0; i < ADC_BUF_SIZE; i++)
        {
            sum += adc_buffer[i];
        }

        adc_average = sum / ADC_BUF_SIZE;
        timer_flag = 1;
    }
}

static int32_t read_bmi160_axis(uint8_t reg_low)
{
    int16_t accel_raw;
    int32_t value;

```

```

I2CStart(0x69, WRITE, 1);
I2CWrite(reg_low);
I2CReStart(0x69, READ, 2);

response = I2CRead();
accel_raw = response;

response = I2CRead();
accel_raw = accel_raw + (response << 8);

I2CStop();

value = accel_raw;
value = (value * 981) / 16384;

return value;
}

static int wrap_led(int n)
{
    while (n < 0) n += NUM_LEDS;
    while (n >= NUM_LEDS) n -= NUM_LEDS;

    return n;
}

static int get_pointer_led(int32_t x, int32_t y, int32_t z)
{
    int32_t mag2_xy = x * x + y * y;

    if (mag2_xy < (DEAD_ZONE * DEAD_ZONE))
    {
        return TOP_LED;
    }

    int best_led = 0;
    int32_t best_dot = -2147483647;

    for (int i = 0; i < NUM_LEDS; i++)
    {
        int32_t dot = x * dir_x[i] + y * dir_y[i];

        if (dot > best_dot)
        {
            best_dot = dot;
            best_led = i;
        }
    }
}

```



```

    return wrap_led(TOP_LED + best_led);
}

static void show_live_and_saved(int live_centre_led, int saved_centre_led, int
saved_active,
                                uint8_t g, uint8_t r, uint8_t b)
{
    int live_a = wrap_led(live_centre_led - 1);
    int live_b = wrap_led(live_centre_led);
    int live_c = wrap_led(live_centre_led + 1);

    int save_a = wrap_led(saved_centre_led - 1);
    int save_b = wrap_led(saved_centre_led);
    int save_c = wrap_led(saved_centre_led + 1);

    ws2812_reset();

    for (int i = 0; i < NUM_LEDS; i++)
    {
        uint8_t gg = 0x00;
        uint8_t rr = 0x00;
        uint8_t bb = 0x00;

        if (saved_active && (i == save_a || i == save_b || i == save_c))
        {
            gg = 0x00;
            rr = 0x00;
            bb = 0x08;
        }

        if (i == live_a || i == live_b || i == live_c)
        {
            gg = g;
            rr = r;
            bb = b;
        }

        ws2812_send_pixel(gg, rr, bb);
    }

    ws2812_reset();
}

void setup(void)
{
    RCC->AHB2ENR |= (1 << 0) | (1 << 1);
    initI2C();
}

```

```

}

void delay(volatile uint32_t dly)
{
    while (dly--) {}
}

void initSerial(uint32_t baudrate)
{
    RCC->AHB2ENR |= (1 << 0);

    pinMode(GPIOA, 2, 2);
    selectAlternateFunction(GPIOA, 2, 7);

    pinMode(GPIOA, 15, 2);
    selectAlternateFunction(GPIOA, 15, 3);

    RCC->APB1ENR1 |= (1 << 17);

    USART2->CR1 = 0;
    USART2->CR2 = 0;
    USART2->CR3 = (1 << 12);
    USART2->BRR = 80000000 / baudrate;

    USART2->CR1 = (1 << 3);
    USART2->CR1 |= (1 << 2);
    USART2->CR1 |= (1 << 0);
}

int _write(int file, char *data, int len)
{
    if ((file != STDOUT_FILENO) && (file != STDERR_FILENO))
    {
        errno = EBADF;
        return -1;
    }

    int original_len = len;

    while (len--)
    {
        eputc(*data);
        data++;
    }

    return original_len;
}

```

```

void eputc(char c)
{
    while ((USART2->ISR & (1 << 6)) == 0) {}
    USART2->TDR = c;
}

static void clock_init_80mhz(void)
{
    RCC->CR |= RCC_CR_HSION;
    while (!(RCC->CR & RCC_CR_HSIRDY)) {}

    FLASH->ACR |= FLASH_ACR_LATENCY_4WS;

    RCC->PLLCFGR =
        RCC_PLLCFGR_PLLSRC_HSI |
        (0 << RCC_PLLCFGR_PLLM_Pos) |
        (10 << RCC_PLLCFGR_PLLN_Pos) |
        (0 << RCC_PLLCFGR_PLLR_Pos) |
        RCC_PLLCFGR_PLLREN;

    RCC->CR |= RCC_CR_PLLON;
    while (!(RCC->CR & RCC_CR_PLLRDY)) {}

    RCC->CFGR &= ~RCC_CFGR_SW;
    RCC->CFGR |= RCC_CFGR_SW_PLL;

    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL) {}

    SystemCoreClock = 80000000;
}

static void gpio_init_ws2812(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

    GPIOA->MODER &= ~(3U << (WS_PIN * 2));
    GPIOA->MODER |= (1U << (WS_PIN * 2));

    GPIOA->OTYPER &= ~(1U << WS_PIN);

    GPIOA->OSPEEDR &= ~(3U << (WS_PIN * 2));
    GPIOA->OSPEEDR |= (3U << (WS_PIN * 2));

    GPIOA->PUPDR &= ~(3U << (WS_PIN * 2));

    GPIOA->BSRR = (1U << (WS_PIN + 16));
}

```

```

static void gpio_init_button(void)
{
    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;

    GPIOB->MODER &= ~(3U << (BTN_PIN * 2));

    GPIOB->PUPDR &= ~(3U << (BTN_PIN * 2));
    GPIOB->PUPDR |= (1U << (BTN_PIN * 2));
}

static int button_pressed(void)
{
    if ((GPIOB->IDR & (1U << BTN_PIN)) == 0)
        return 1;
    else
        return 0;
}

static void dwt_init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}

static void delay_us(uint32_t us)
{
    uint32_t start = DWT->CYCCNT;
    uint32_t ticks = us * (SystemCoreClock / 1000000U);

    while ((DWT->CYCCNT - start) < ticks) {}
}

static void ws2812_reset(void)
{
    GPIOA->BSRR = (1U << (WS_PIN + 16));
    delay_us(80);
}

static void ws2812_send_bit(uint8_t bit)
{
    uint32_t start = DWT->CYCCNT;

    if (bit)
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 64) {}
    }
}

```

```

        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
    else
    {
        GPIOA->BSRR = (1U << WS_PIN);
        while ((DWT->CYCCNT - start) < 32) {}

        GPIOA->BSRR = (1U << (WS_PIN + 16));
        while ((DWT->CYCCNT - start) < 100) {}
    }
}

static void ws2812_send_byte(uint8_t byte)
{
    for (int i = 7; i >= 0; i--)
    {
        ws2812_send_bit((byte >> i) & 0x01);
    }
}

static void ws2812_send_pixel(uint8_t g, uint8_t r, uint8_t b)
{
    ws2812_send_byte(g);
    ws2812_send_byte(r);
    ws2812_send_byte(b);
}

static void initTimer2PWM(void)
{
    RCC->APB1ENR1 |= (1 << 0);

    TIM2->CR1 = 0;
    TIM2->CCMR2 = (0b110 << 12) | (1 << 11);
    TIM2->CCER |= (1 << 12);

    TIM2->PSC = 79;
    TIM2->ARR = 1000 - 1;
    TIM2->CCR4 = 0;

    TIM2->EGR |= (1 << 0);
    TIM2->CR1 |= (1 << 7);
    TIM2->CR1 |= (1 << 0);
}

static void setTimer2Duty(int duty)
{
    int arrvalue;

```

```

    if (duty < 0)
        duty = 0;

    if (duty > 4095)
        duty = 4095;

    if (duty <= SAVE_THRESHOLD)
    {
        TIM2->CCR4 = 0;
        return;
    }

    arrvalue = (duty * (TIM2->ARR + 1)) / 4095;
    TIM2->CCR4 = arrvalue;
}

```

Result: The final integrated system successfully retained all original Project 1 functionality while adding a tilt-triggered audio warning feature. The WS2812 LED ring continued to display the live tilt direction using a 3-LED pointer, colour selection through the push button remained operational, PWM brightness control from the potentiometer functioned correctly, and saved-position mode still operated as intended.

A 300 Hz warning tone was produced through the MAX98357A speaker whenever excessive tilt was detected in the X or Y direction. The tone activated once the filtered accelerometer values exceeded the defined AUDIO_LIMIT threshold of 750. Audio generation used the SAI peripheral configured in I²S mode, with DMA2 Channel 6 continuously transferring samples from the audio_data waveform table to the SAI data register in circular mode. The tone_silent buffer was played during normal operation, muting the speaker when tilt was within the safe range. Switching between the warning tone and the silent buffer was handled by set_audio_tone(), which disabled the DMA channel and SAI block, updated the memory address and transfer count, then re-enabled both.

ADC sampling was performed continuously using DMA1 Channel 1 in circular mode, filling a 32-sample buffer from the potentiometer on PA0. TIM6 generated a periodic interrupt using TIM6_DAC_IRQHandler which averaged the ADC buffer and set a flag for the main loop to print the current ADC value over UART.

The button was relocated to PB3 for this task to avoid a pin conflict with the SAI audio lines occupying PA8, PA9 and PA10. All subsystems — I²C accelerometer, WS2812 LED ring, SAI/I²S audio, DMA ADC sampling, TIM2 PWM, TIM6 interrupt, and UART debugging — operated simultaneously without timing conflicts.

